24BCE1270
Manan Dudeja

Design And Analysis of Algorithms
Inlab practice 12- Bellman Ford

- ## Logic-

Bellman-Ford algorithm computes the shortest path from a single source vertex to all other vertices in a weighted graph.

It works by relaxing all edges repeatedly.

Time Complexity:

- Worst Case: O(V × E)
- Can detect negative weight cycles.

- ## Pseudocode-

```
Initialize distance[source] = 0
Initialize all other distances = infinity

Repeat V-1 times:
   For each edge (u,v,w):
      If distance[u] + w < distance[v]
         distance[v] = distance[u] + w

Check for negative cycles:
   For each edge (u,v,w):
      If distance[u] + w < distance[v]
         Print "Negative cycle exists"
```

- **Cpp Code:**

```
#include <iostream>
#include <vector>
#include <fstream>
#include <ctime>

using namespace std;

struct Edge {
   int u, v, w;
};

void bellmanFord(int V, int E, vector<Edge>& edges, int src) {

   const int INF = 1000000;
   vector<int> dist(V, INF);

   dist[src] = 0;

   for(int i = 1; i <= V-1; i++) {

       for(int j = 0; j < E; j++) {

           int u = edges[j].u;
           int v = edges[j].v;
           int w = edges[j].w;

           if(dist[u] != INF && dist[u] + w < dist[v])
               dist[v] = dist[u] + w;
       }
```

```
    }

    for(int j = 0; j < E; j++) {
```

```
        int u = edges[j].u;
        int v = edges[j].v;
        int w = edges[j].w;

        if(dist[u] != INF && dist[u] + w < dist[v]) {
            cout << "Graph contains negative weight cycle\n";
            return;
        }
    }

    cout << "Shortest distances:\n";

    for(int i = 0; i < V; i++)
        cout << "Vertex " << i << " : " << dist[i] << endl;
}

int main(int argc, char* argv[]) {

    if(argc < 2) {
        cout << "Usage: ./a.out inputfile\n";
        return 0;
    }

    ifstream file(argv[1]);

    if(!file) {
```

```
        cout << "File not found\n";
        return 0;
    }

    int V, E;
    file >> V >> E;

    vector<Edge> edges(E);
```

```
    for(int i = 0; i < E; i++)
        file >> edges[i].u >> edges[i].v >> edges[i].w;

    clock_t start = clock();

    bellmanFord(V, E, edges, 0);

    clock_t end = clock();

    double timeTaken = double(end - start) / CLOCKS_PER_SEC;

    cout << "Time Taken: " << timeTaken << " seconds\n";

    return 0;
}
```

- **Python Code:**
  - Average case code:

```
import random


sizes = [10, 20, 50, 100, 200]
```

```python
for n in sizes:

    filename = "bellman_" + str(n) + ".txt"

    edges = n * 3

    with open(filename, "w") as f:

        f.write(str(n) + " " + str(edges) + "\n")

        for i in range(n-1):
            w = random.randint(1, 20)
            f.write(f"{i} {i+1} {w}\n")

        for i in range(edges - (n-1)):
            u = random.randint(0, n-1)
            v = random.randint(0, n-1)
            w = random.randint(1, 20)
            f.write(f"{u} {v} {w}\n")

    print("Generated:", filename)
```

Instructions to check runtime:

1. Create a folder on Desktop.
2. Save:

- C++ file →bellman.cpp
- Python file →case.py

3. Open terminal in the folder. 4. Generate input files:

python3 case.py

5. Compile C++ program:

```
g++ bellman.cpp
```

6. Run program:

```
./a.out bellman_200.txt
```

7. Repeat for different input sizes.
8. Note down execution time.
9. Plot Input Size vs Time Taken graph in MS Excel.

Output:

```
student@ab1205b-50:~/Downloads/hello$ g++ bellman.cpp
student@ab1205b-50:~/Downloads/hello$ ./a.out bellman_10.txt
Shortest distances:
Vertex 0 : 0
Vertex 1 : 14
Vertex 2 : 26
Vertex 3 : 25
Vertex 4 : 28
Vertex 5 : 28
Vertex 6 : 1
Vertex 7 : 2
Vertex 8 : 17
Vertex 9 : 27
Time Taken: 6.6e-05 seconds
```

```
student@ab1205b-50:~/Downloads/hello$ ./a.out bellman_20.txt
Shortest distances:
Vertex 0 : 0
Vertex 1 : 18
Vertex 2 : 26
Vertex 3 : 34
Vertex 4 : 52
Vertex 5 : 51
Vertex 6 : 27
Vertex 7 : 13
Vertex 8 : 8
Vertex 9 : 14
Vertex 10 : 6
Vertex 11 : 9
Vertex 12 : 18
Vertex 13 : 22
Vertex 14 : 29
Vertex 15 : 32
Vertex 16 : 10
Vertex 17 : 26
Vertex 18 : 40
Vertex 19 : 37
Time Taken: 8.1e-05 seconds
```

```
student@ab1205b-50:~/Downloads/hello$ ./a.out bellman_50.txt
Shortest distances:
Vertex 0 : 0
Vertex 1 : 15
Vertex 2 : 25
Vertex 3 : 20
Vertex 4 : 22
Vertex 5 : 25
Vertex 6 : 29
Vertex 7 : 30
Vertex 8 : 28
Vertex 9 : 47
Vertex 10 : 45
Vertex 11 : 65
Vertex 12 : 66
Vertex 13 : 53
Vertex 14 : 44
```

.

. .

```
Vertex 34 : 24
Vertex 35 : 30
Vertex 36 : 40
Vertex 37 : 35
Vertex 38 : 36
Vertex 39 : 28
Vertex 40 : 30
Vertex 41 : 45
Vertex 42 : 65
Vertex 43 : 27
Vertex 44 : 28
Vertex 45 : 28
Vertex 46 : 31
Vertex 47 : 37
Vertex 48 : 36
Vertex 49 : 50
Time Taken: 0.000344 seconds
```

```
student@ab1205b-50:~/Downloads/hello$ ./a.out bellman_100.txt
Shortest distances:
Vertex 0 : 0
Vertex 1 : 3
Vertex 2 : 18
Vertex 3 : 19
Vertex 4 : 25
Vertex 5 : 38
Vertex 6 : 40
Vertex 7 : 53
Vertex 8 : 30
Vertex 9 : 48
Vertex 10 : 37
Vertex 11 : 48
Vertex 12 : 38
Vertex 13 : 33
Vertex 14 : 33
```

.

. .

```
Vertex 90 : 45
Vertex 91 : 55
Vertex 92 : 19
Vertex 93 : 36
Vertex 94 : 20
Vertex 95 : 23
Vertex 96 : 30
Vertex 97 : 47
Vertex 98 : 39
Vertex 99 : 54
Time Taken: 0.000916 seconds
```

```
student@ab1205b-50:~/Downloads/hello$ ./a.out bellman_200.txt
Shortest distances:
Vertex 0 : 0
Vertex 1 : 15
Vertex 2 : 26
Vertex 3 : 30
Vertex 4 : 47
Vertex 5 : 51
Vertex 6 : 63
Vertex 7 : 37
Vertex 8 : 37
Vertex 9 : 34
Vertex 10 : 42
Vertex 11 : 44
```

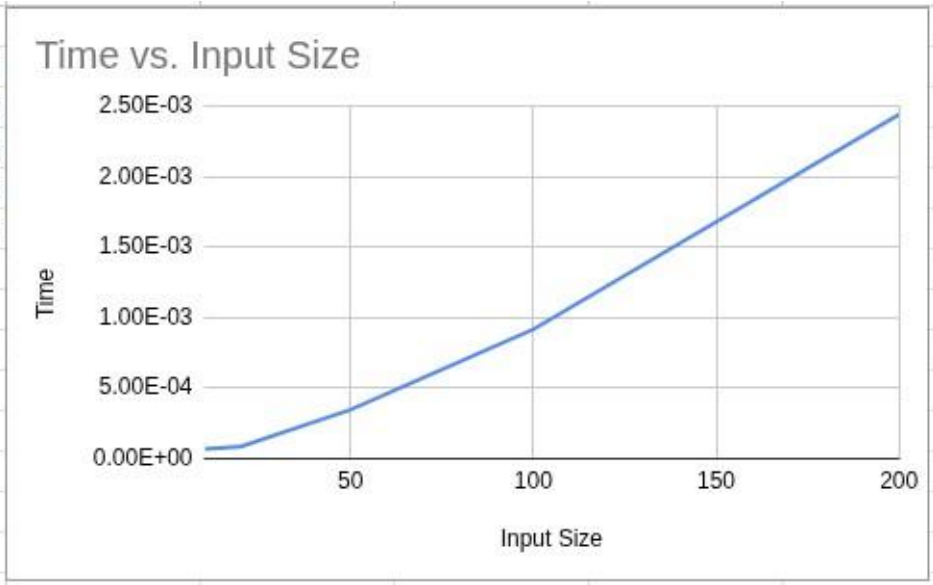
.

. .

```
Vertex 190 : 46
Vertex 191 : 40
Vertex 192 : 46
Vertex 193 : 61
Vertex 194 : 39
Vertex 195 : 48
Vertex 196 : 54
Vertex 197 : 56
Vertex 198 : 60
Vertex 199 : 59
Time Taken: 0.002441 seconds
```

Graph-

Input Size	Time
10	6.60E-05
20	8.10E-05
50	0.000344
100	0.000916
200	0.002441



- **<u>Result-</u>**

Bellman-Ford algorithm successfully computes shortest paths from a single source even when negative edge weights are present.

Its time complexity is O(VE), which makes it slower for very large graphs.